Name : Saswata Kumar Nayak **Mail id : 22052504@kiit.ac.in**

Superset id : 6363187 **University Roll : 22052504**

Week - 06

1. Creating First react app

- Define SPA and its benefits
- Define React and identify its working
- Identify the differences between SPA and MPA
- Explain Pros & Cons of Single-Page Application
- Explain about React
- Define virtual DOM
- Explain Features of React

1. SPA (Single Page Application)

Loads one HTML page and updates content dynamically.

Benefits: Fast, smooth user experience, reduced server load.

2. React

A JavaScript library for building user interfaces.

Component-based, fast updates using virtual DOM.

3. SPA vs MPA

SPA MPA

Loads once Loads new page each time

Fast Slower

Uses JS routing Server routing

SEO is harder SEO is easier

4. SPA Pros & Cons

Pros: Fast, app-like feel, efficient.

Cons: SEO issues, initial load can be heavy.

5. React Overview

Builds UIs using reusable components.

Uses JSX and unidirectional data flow.

6. Virtual DOM

A lightweight copy of the real DOM.

React updates only the changed parts → faster performance.

7. Features of React

JSX syntax

Virtual DOM

Component-based

One-way data flow

Fast rendering

Strong community support

Output:-

Welcome to the first session of React

---

2. Creating a react app for Student Management Portal -

Objectives

- Explain React components
- Identify the differences between components and JavaScript functions
- Identify the types of components
- Explain class component
- Explain function component
- Define component constructor
- Define render() function

1. React Components

Building blocks of a React UI.

Reusable pieces that return JSX (HTML + JS).

2. Components vs JavaScript Functions

React Component	JavaScript Function
Returns JSX	Returns values/data
Part of UI	General logic
Has lifecycle	No lifecycle

3. Types of Components

Class Component (older style)

Function Component (modern and preferred)

4. Class Component

Uses class syntax.

Has lifecycle methods and state.

```
class MyComponent extends React.Component {
  render() {
    return <h1>Hello</h1>;
  }
}
```

5. Function Component

Simple function that returns JSX.

Uses Hooks for state and lifecycle.

```
function MyComponent() {
  return <h1>Hello</h1>;
}
```

6. Component Constructor

Special method in class components.

Initializes state and binds methods.

```
constructor(props) {
  super(props);
```

```
  this.state = { count: 0 };
}
```

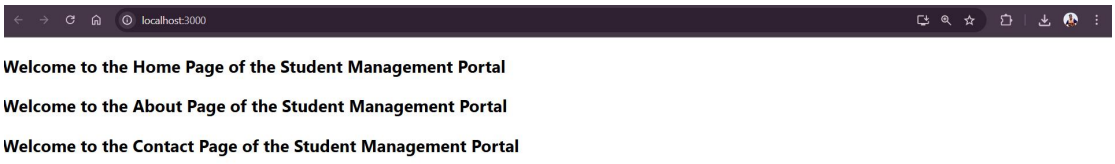
7. render() Function

Required in class components.

Returns JSX to display the UI.

```
render() {
  return <div>UI goes here</div>;
}
```

Output :



---

3. Creating a react app for Student Management Portal - Calculate score

Objectives

- Explain React components
- Identify the differences between components and JavaScript functions
- Identify the types of components
- Explain class component
- Explain function component
- Define component constructor
- Define render() function

1. React Components

Core building blocks of React UI.

Return JSX to define how UI looks.

2. Components vs JavaScript Functions

React Components	JavaScript Functions
Return JSX/UI	Return data or logic
Can hold state	Stateless
Have lifecycle methods	No lifecycle

3. Types of Components

Class Components

Function Components

4. Class Component

Uses class syntax.

Supports state and lifecycle methods.

```
class MyComp extends React.Component {
  render() {
    return <h1>Hello</h1>;
  }
}
```

5. Function Component

Uses simple function syntax.

Cleaner and supports hooks for state and effects.

```
function MyComp() {
  return <h1>Hello</h1>;
}
```

6. Component Constructor

Used in class components.

Initializes state and binds methods.

```
constructor(props) {
  super(props);
  this.state = { count: 0 };
}
```

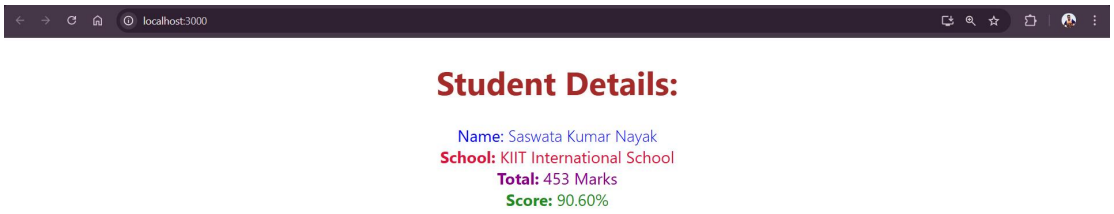
7. render() Function

Required in class components.

Returns JSX to be shown on the screen.

```
render() {
  return <div>Hello World</div>;
}
```

Output :



---

4. Blogapp

Objectives

- Explain the need and Benefits of component life cycle
- Identify various life cycle hook methods
- List the sequence of steps in rendering a component

1. Need & Benefits of Component Lifecycle

Why it's needed:

To manage what happens when a component is created, updated, or destroyed.

Benefits:

Control over rendering.

Efficient resource management (like timers, API calls).

Helps in debugging and optimization.

Smooth integration with external libraries.

2. Lifecycle Hook Methods (in Class Components)

Phase	Method	Purpose
Mounting	constructor()	Initialize state
	componentDidMount()	Called after component is added
Updating	shouldComponentUpdate()	Decide if re-render is needed
	componentDidUpdate()	Runs after update
Unmounting	componentWillUnmount()	Cleanup before removal

In Function Components, lifecycle is managed using Hooks like:

useEffect() → replaces componentDidMount, componentDidUpdate, and componentWillUnmount.

3. Sequence of Steps in Rendering a Component (Class)

Mounting phase:

constructor()

render()

componentDidMount()

Updating phase:

shouldComponentUpdate()

reder()

componentDidUpdate()

Unmounting phase:

componentWillUnmount()

Output :

5. Cohort Tracker

Objectives

- Understanding the need for styling react component
- Working with CSS Module and inline styles

1. Need for Styling React Components

Why style React components?

To make the UI visually appealing and user-friendly.

To maintain consistent design across components.

To reflect dynamic states (hover, active, etc.).

Benefits:

Better user experience

Branding and visual identity

Improved readability and structure

2. Working with CSS Modules

CSS Modules provide scoped styles to components (avoids class name conflicts).

File must end with .module.css.

Example:

```
/* styles.module.css */
.title {
  color: green;
}
```

```
import styles from './styles.module.css';
function MyComponent() {
  return <h1 className={styles.title}>Hello</h1>;
}
```

3. Working with Inline Styles

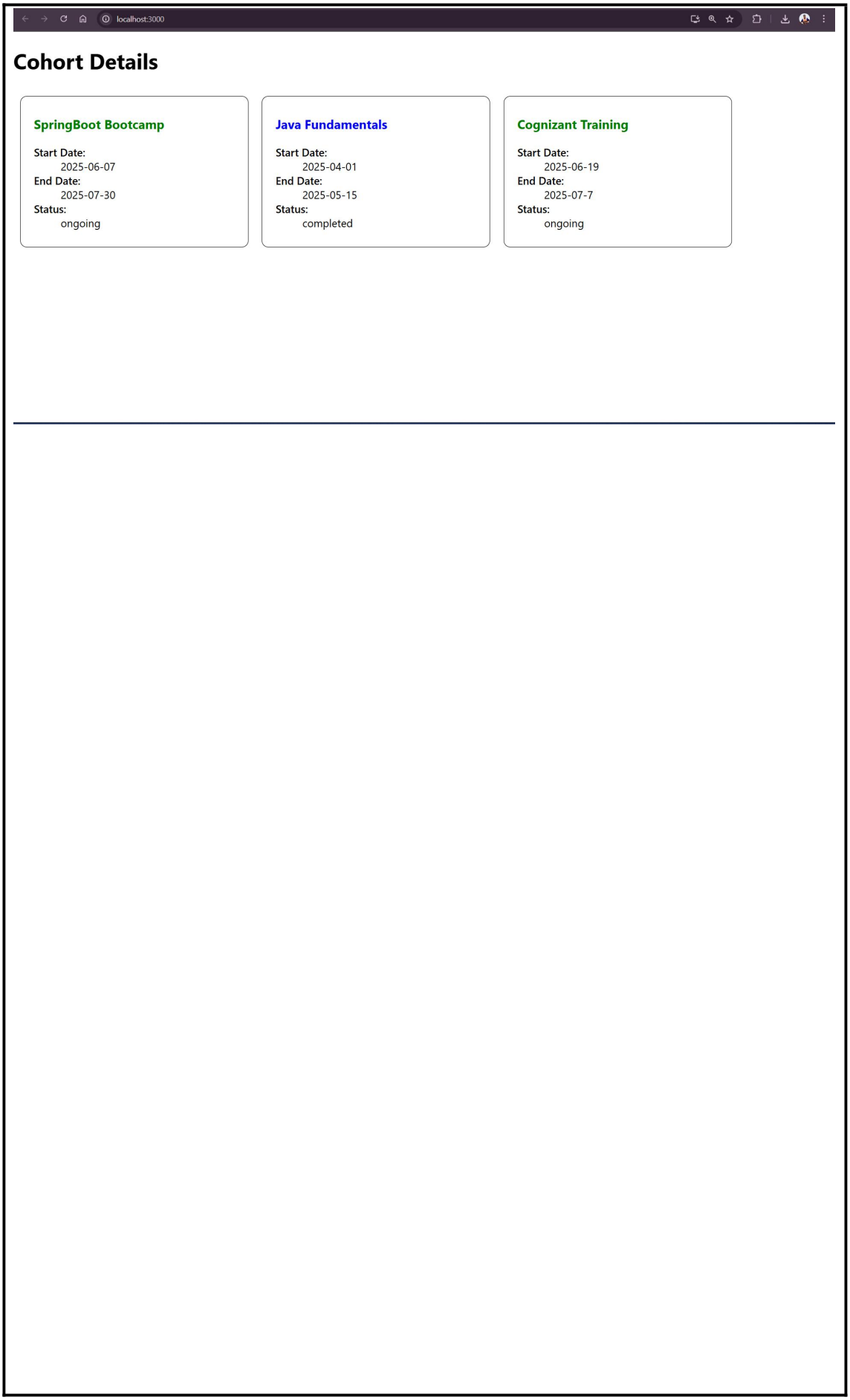
Defined as JavaScript objects.

Style names use camelCase instead of kebab-case.

Example:

```
const headingStyle = {
  color: 'blue',
  fontSize: '24px'
};
function MyComponent() {
  return <h1 style={headingStyle}>Welcome</h1>;
}
```

Output :


localhost:3000
Cohort Details
SpringBoot Bootcamp
Start Date:
2025-06-07
End Date:
2025-07-30
Status:
ongoing
Java Fundamentals
Start Date:
2025-04-01
End Date:
2025-05-15
Status:
completed
Cognizant Training
Start Date:
2025-06-19
End Date:
2025-07-7
Status:
ongoing